
TYPESCRIPT OOP

Mastering **Object-Oriented Programming** with
TypeScript

Object-Oriented Programming (OOP) is a paradigm based on "objects" containing data and code. TypeScript enhances this with static typing and robust class-based features.

CLASSES & OBJECTS

THE BLUEPRINT

A class is a blueprint for creating objects, providing initial values for state and implementations of behavior.

THE INSTANCE

An object is a concrete entity created from the class blueprint, known as an instance.

PROPERTIES & METHODS

Properties hold data associated with an object, while methods define its behavior.

```
class Car {
  make: string;
  model: string;

  constructor(make: string, model: string) {
    this.make = make;
    this.model = model;
  }

  displayInfo(): void {
    console.log(`Car: ${this.make} ${this.model}`);
  }
}

const myCar = new Car("Toyota", "Camry");
myCar.displayInfo(); // Output: Car: Toyota Camry
```

ENCAPSULATION

DATA PROTECTION

Encapsulation bundles data and methods into a single unit, restricting direct access to prevent accidental misuse.

- PUBLIC** Accessible from anywhere (Default)
- PRIVATE** Accessible only within the class
- PROTECTED** Accessible within class and subclasses

GETTERS & SETTERS

Provide controlled access to private properties, allowing validation and logic during data retrieval or modification.

```
class BankAccount {
  private _balance: number;

  constructor(initial: number) {
    this._balance = initial;
  }

  deposit(amount: number): void {
    if (amount > 0) this._balance += amount;
  }

  get balance(): number {
    return this._balance;
  }
}

const acc = new BankAccount(1000);
acc.deposit(500);
console.log(acc.balance); // 1500
// acc._balance = 0; // Error: private
```

INHERITANCE

THE "IS-A" RELATIONSHIP

Inheritance allows a class to inherit properties and methods from another class, promoting code reuse and hierarchy.

EXTENDING CLASSES

Use the **extends** keyword to create a subclass. The subclass gains all non-private members of the superclass.

THE SUPER KEYWORD

The **super** keyword is used to call the constructor of the parent class and access its methods.

METHOD OVERRIDING

Subclasses can provide a specific implementation for a method already defined in the superclass.

```
class Animal {
    constructor(public name: string) {}
    move(dist: number) {
        console.log(`${this.name} moved ${dist}m.`);
    }
}

class Dog extends Animal {
    constructor(name: string, public breed: string) {
        super(name); // Call parent constructor
    }

    bark() { console.log("Woof!"); }

    // Overriding move method
    move(dist: number = 5) {
        console.log("Running...");
        super.move(dist);
    }
}

const myDog = new Dog("Buddy", "Lab");
myDog.move();
```

ABSTRACTION

HIDING COMPLEXITY

Abstraction focuses on "what" an object does rather than "how" it does it, hiding complex implementation details.

ABSTRACT CLASSES

Classes that cannot be instantiated directly and serve as base classes, often containing abstract methods.

INTERFACES

Define contracts that objects must follow, specifying properties and methods without implementation.

```
abstract class Shape {
    abstract getArea(): number;
}

class Circle extends Shape {
    constructor(private radius: number) { super(); }
    getArea(): number {
        return Math.PI * this.radius ** 2;
    }
}

interface Drivable {
    start(): void;
}

class Car implements Drivable {
    start() { console.log("Engine started"); }
}
```

POLYMORPHISM

"MANY FORMS"

Polymorphism allows objects of different classes to be treated as objects of a common superclass or interface.

INTERFACE-BASED

The most powerful form in TypeScript. Different classes implementing the same interface can be used interchangeably.

METHOD OVERRIDING

Subclasses provide specific implementations of methods defined in their parent class, allowing for dynamic behavior at runtime.

```
interface Greetable {
  greet(): string;
}

class English implements Greetable {
  greet() { return "Hello!"; }
}

class Spanish implements Greetable {
  greet() { return "¡Hola!"; }
}

function sayHello(person: Greetable) {
  console.log(person.greet());
}

const users: Greetable[] = [
  new English(),
  new Spanish()
];

users.forEach(sayHello);
// Output: Hello!, ¡Hola!
```

E-COMMERCE SYSTEM

PUTTING IT ALL TOGETHER

This example combines all OOP principles to model a basic e-commerce system with products and a shopping cart.

- INTERFACES FOR CONTRACTS
- ENCAPSULATED STATE
- CLASS INHERITANCE
- POLYMORPHIC BEHAVIOR

Notice how both BaseProduct and ElectronicProduct are treated uniformly as 'Product' in the cart.

```
interface Product {
  id: string; price: number;
  getDescription(): string;
}

class BaseProduct implements Product {
  constructor(public id: string, protected _price: number) {}
  get price() { return this._price; }
  getDescription() { return `ID: ${this.id} - ${this._price}`; }
}

class Electronic extends BaseProduct {
  constructor(id: string, price: number, public brand: string) {
    super(id, price);
  }
  getDescription() { return `${super.getDescription()} by ${this.brand}`; }
}

class Cart {
  private items: Product[] = [];
  addItem(p: Product) { this.items.push(p); }
  getTotal() { return this.items.reduce((s, i) => s + i.price, 0); }
}

const cart = new Cart();
cart.addItem(new Electronic("E1", 1000, "Dell"));
cart.addItem(new BaseProduct("B1", 20));
```

CONCLUSION

RECAP OF PILLARS

Mastering Encapsulation, Inheritance, Abstraction, and Polymorphism is essential for building robust TypeScript applications.

WHY TYPESCRIPT?

Static typing and interfaces provide the safety and structure needed to implement OOP patterns effectively at scale.

NEXT STEPS

Explore Design Patterns (SOLID, Creational, Structural) and apply these principles in your daily development workflow.

BEST PRACTICES

- 01** Favor composition over inheritance for greater flexibility.
- 02** Use interfaces to define clear contracts between components.
- 03** Keep classes small and focused on a single responsibility.
- 04** Encapsulate internal state using private/protected modifiers.
- 05** Leverage static typing to catch errors during development.